Implementation and Evaluation of Dijkstra's Algorithm with Different Priority Queues

2026-10-28

Abstract

This project compares seven priority-queue strategies for Dijkstra's algorithm within a shared CSR graph representation and queue interface. The comparison includes a Fibonacci heap and also considers FIFO SPFA and SLF+LLL SPFA, although neither SPFA variant is a heap-based form of Dijkstra's algorithm. The experiments cover 171 random-graph configurations, 126 fixed-density configurations, and 1,000 queries on each of the NY and BAY DIMACS road networks for nine implementations. In the WSL2 Release measurements, the radix heap was the fastest Dijkstra queue on both road networks, with mean wall-clock times of 12.621 and 13.873 ms/query for NY and BAY, respectively. SLF+LLL SPFA required 7.267 and 9.258 ms/query. The Fibonacci heap showed no practical speed advantage on either network. The report distinguishes asymptotic guarantees, finite-size model fits, and measured runtimes, and it states the applicable range of the observed results.

Keywords: Dijkstra's algorithm; Fibonacci heap; radix heap; DIMACS; complexity analysis

Contents

Chapter 1 Introduction **3**

1.1 Problem Description and Objectives . . . 3
1.2 Implementations and Experimental Scope . . . 3
1.3 Experimental Setup . . . 3

Chapter 2 Data Structure / Algorithm Specification **4**

2.1 CSR Graph and Shared Interface . . . 4
2.2 Main Dijkstra Loop . . . 4
2.3 Binary Heap, Segment Tree, and PBDS . . . 4
2.4 Fibonacci Heap . . . 5
2.5 Monotonic Radix Heap . . . 6
2.6 Two Label-Correcting Algorithms . . . 6

Chapter 3 Testing Results **7**

3.1 Experimental Environment and Timing Methodology . . . 7
3.2 USA Road Network Inputs and Results . . . 7
3.3 Random Graph Parameter Sweep . . . 9
3.4 Fixed-Density Experiments and Complexity Fitting . . . 10

Chapter 4 Analysis and Comments **12**

4.1 Time and Space Complexity . . . 12
4.2 Observed Performance of the Fibonacci Heap . . . 13
4.3 Differences between the Radix Heap and SPFA . . . 14
4.4 Model Fitting and Measurement Limitations . . . 14
4.5 Further Improvements . . . 15

Appendix A Source Code and Reproduction **16**

A.1 Source Code Organization . . . 16
A.2 Random and Fixed-Density Graph Reachability Results . . . 17

References **19**

Declaration **19**

Chapter 1 Introduction

1.1 Problem Description and Objectives

Given a directed graph $G = (V, E)$, nonnegative edge weights $w : E \to \mathbb{Z}_{\geq 0}$, and a source vertex s, the single-source shortest-path problem asks for the shortest distance $d[v]$ from s to every vertex. The distance to an unreachable vertex is denoted by ∞. Let $n = |V|$ and $m = |E|$; throughout this report, m refers to the number of directed arcs.

Dijkstra's greedy order depends on a minimum-priority queue. Binary heaps, segment trees, and Fibonacci heaps handle insertion, decrease-key, and extract-min differently, which leads to different asymptotic costs and implementation constants. When distances are nonnegative integers, a monotonic radix heap is another option.

The study addresses two questions: whether the lower theoretical cost of decrease-key operations translates into faster searches, and how the speed ordering of the implementations changes with graph structure.

1.2 Implementations and Experimental Scope

We implemented and compared seven queues for Dijkstra's algorithm: the standard library priority queue, a custom binary heap, a segment-tree heap, a Fibonacci heap, a radix heap, the GNU PBDS pairing heap, and the GNU PBDS thin heap. Except for the standard-library version, which simulates decrease-key by inserting duplicate entries, the implementations support locating a vertex and updating its key. FIFO SPFA and SLF+LLL SPFA are queue-based label-correcting algorithms rather than heap variants of Dijkstra's algorithm, and they are treated separately in this report.

All implementations share the main loop in `general-dijkstra.hpp` and the same CSR graph representation. Random-graph and fixed-density experiments use seed 42 and the same graph-generation rule for each parameter set. The road-network experiments use `USA-road-d.NY` and `USA-road-d.BAY`, with 1,000 generated random source-target queries and seed 971.

1.3 Experimental Setup

Although the road-network input is expressed as point-to-point queries, the current algorithms compute the complete distance array from each source and drain the queue. The target vertex does not trigger early termination. Consequently, the reported "ms/query" is **the mean time for a complete single-source search triggered by one source-target pair**; it is not the latency of a point-to-point search that stops at the target. This study does not include path recovery, preprocessing indices, or parallel search.

Random-graph and fixed-density experiments are repeated three times for each configuration. Each road-network implementation runs one complete batch containing 1,000 queries.

Chapter 2 Data Structure / Algorithm Specification

2.1 CSR Graph and Shared Interface

Vertices are numbered $1, \ldots, n$. The CSR representation stores a contiguous edge array and an offset array of length $n+1$, grouping arcs by their source. Visiting the outgoing arcs of a vertex therefore takes time proportional to its out-degree. Each edge stores a destination and an integer weight, and shortest distances use signed 64-bit integers. The initial infinity value is set to one quarter of the maximum value of this type, which leaves headroom for distance addition over the inputs used here. The implementation does not provide general overflow detection for arbitrarily large inputs.

The priority-queue interface provides `push_or_decrease(key,v)`, `pop()`, `empty()`, and `clear()`. Implementations that support decrease-key use the vertex index to locate an entry, while a shared base class uses timestamps to identify the current in-heap entry. `pop()` clears the corresponding in-heap marker. The queue must be empty at the end of a complete search and can then be reused.

2.2 Main Dijkstra Loop

The source distance is initialized to zero, and each relaxation updates a label using the length of an already discovered path. The standard-library version may contain stale keys, so the main loop skips outdated entries.

```
DIJKSTRA(G, s, Q):
  fill d[0..n] with INF
  d[s] = 0; Q.push_or_decrease(0, s)
  while not Q.empty():
    (key, u) = Q.pop()
    if key > d[u]: continue
    for each (v, w) in G[u]:
      if d[u] + w < d[v]:
        d[v] = d[u] + w
        Q.push_or_decrease(d[v], v)
  return d
```

Listing 1: Shared Dijkstra pseudocode

2.3 Binary Heap, Segment Tree, and PBDS

Standard-library priority queue. The minimum heap stores (d, v) pairs. Each distance improvement inserts a new entry, and outdated entries are discarded when removed. The heap may therefore contain multiple keys for the same vertex, and its space usage grows with the number of successful relaxations.

Custom binary heap. The array `heap[pos]` stores vertices, `idx[v]` stores positions, and `key[v]` stores the current keys. Insertion places a vertex at the end and sifts it upward; decrease-key starts from `idx[v]` and also sifts upward. Extract-min moves the last vertex to the root and

sifts it downward, with every swap updating the positions of both vertices. Each vertex has at most one entry in the heap.

Segment-tree heap. Each vertex corresponds to a leaf, and each internal node stores the minimum key in its interval and the associated vertex. Insertion and decrease-key update the path to the root. Extract-min reads the vertex stored at the root and sets its leaf to ∞. Even when the minimum key itself does not change, propagation must continue if the minimizing vertex changes, preventing stale vertices with equal keys. The largest vertex index and the sentinel leaf are kept separate.

PBDS pairing and thin heaps. The adaptor stores a `point_iterator` for each vertex. The first insertion calls `push`, and later updates use `modify`. Internal heap-order maintenance is delegated to the GNU library. This project does not reimplement either library heap and does not equate their decrease-key amortized bounds with that of a Fibonacci heap.

2.4 Fibonacci Heap

Each vertex has a slot in a preallocated node array. A node stores its key, parent, children, left and right sibling pointers, degree, and mark. Both the root list and the child lists are circular doubly linked lists, with a separate pointer to the minimum root. Insertion splices a singleton tree into the root list. Extract-min promotes the children of the minimum root and then links trees of equal degree. The root list is processed by unlinking the current root, which avoids relying on a circular termination point that can become invalid while consolidation is in progress.

```
INSERT(v, key):
  initialize node[v] as a singleton unmarked root
  splice node[v] into root list; update minimum
EXTRACT_MIN():
  remove minimum root z
  promote all children of z to unmarked roots
  while two roots have the same degree:
    link the larger-key root under the smaller-key root
  rebuild root list and minimum; return z
DECREASE(v, key):
  node[v].key = key
  if v violates heap order with its parent:
    cut v to the root list
    mark its parent on the first child loss
    repeatedly cut marked ancestors after another child loss
  update minimum if needed
```

Listing 2: Key operations of the Fibonacci heap

A mark records whether a non-root node has already lost one child. A potential function may be chosen as $\Phi = t + 2q$, where t is the number of roots and q is the number of marked nodes. The reduction in marks caused by cascading cuts pays for the cut operations. The Fibonacci heap therefore supports amortized $O(1)$ insertion and decrease-key, amortized $O(\log n)$ extract-min, and the $O(m + n \log n)$ bound for Dijkstra's algorithm [1]. This bound describes the total sequence of operations; it does not imply that every extract-min takes logarithmic time.

2.5 Monotonic Radix Heap

The radix heap stores the most recently extracted key L. A key k is placed in the bucket numbered $b(k) = \text{bitwidth}(k \text{ xor } L)$; bucket zero contains only keys equal to L. If bucket zero is empty, the smallest key in the lowest nonempty bucket becomes the new L, and that bucket is redistributed according to the new L. Redistribution reduces the bucket indices. Decrease-key uses a handle that records the bucket index and the entry's position within the bucket. A cross-bucket move swaps entries and repairs the handle of the swapped entry.

```
POP():
  if bucket[0] is empty:
    b = smallest nonempty bucket id
    L = minimum key in bucket[b]
    redistribute every entry in bucket[b] using bitwidth(key XOR L)
  remove and return an entry from bucket[0]
  if all buckets are empty: reset L = 0 for the next query
```

Listing 3: Extract-min in the radix heap

A valid insertion requires $k \geq L$; it does not require all inserted keys to be globally increasing. Nonnegative edge weights in Dijkstra's algorithm ensure that a newly discovered candidate distance is no smaller than the most recently extracted valid distance, which satisfies this condition. The current implementation has 32 buckets and uses 25 bits for the position information in each handle. It is not a general radix heap for arbitrary 64-bit key ranges.

2.6 Two Label-Correcting Algorithms

FIFO SPFA. The queue stores (d, v) entries, and each successful relaxation inserts a new entry. Outdated distances are skipped when an entry is removed. There is no separate in-queue marker, so duplicate entries are possible; memory use and operation counts therefore should not be analyzed as if each vertex had exactly one queue entry.

SLF+LLL SPFA. The deque stores vertices and maintains an in-queue marker. When a successful relaxation improves a vertex that is not queued, the vertex is pushed to the front if its label is smaller than the current front label (SLF); otherwise it is pushed to the back. Before processing the front vertex, the algorithm moves it to the back if its label is greater than the average label of queued vertices (LLL). The average is maintained from the sum of queued labels and updated when an already queued label decreases. These heuristics change only the relaxation order and provide no Dijkstra-style finalization guarantee.

```
ENQUEUE(v):
  if queue empty or d[v] < d[front]: push_front(v)
  else: push_back(v)
  in_queue[v] = true; sum += d[v]; count += 1
PROCESS():
  while d[front] * count > sum: rotate front to back
  u = pop_front(); in_queue[u] = false
  sum -= d[u]; count -= 1
  relax outgoing arcs of u
  update sum on an already queued label decrease
  otherwise enqueue each newly improved vertex
```

Listing 4: Queue scheduling rules for SLF+LLL

Chapter 3 Testing Results

3.1 Experimental Environment and Timing Methodology

Table 1: Experimental environment and configuration

Item	Configuration
Platform	Intel Core i9-14900HX; WSL2 Arch Linux with 32 logical CPUs
Toolchain	GCC 16.2.1; CMake 4.4.3; Ninja 1.13.2; Google Benchmark 1.9.5
Compilation	C++20, Release, `-O3 -DNDEBUG`
Input and execution	Binaries and graph data stored on WSL ext4
Execution policy	Benchmarks run serially; random and complexity experiments use three repetitions; each road implementation runs one complete batch
Timing	Automatic iteration target of 0.5 s; each road-network iteration processes 1,000 queries

Graph generation, text parsing, priority-queue construction, and the initial allocation of the distance array are outside the timing loop. Reinitialization of the distance array remains inside each timed search. The local in-queue marker array for SLF+LLL is allocated in its search function and is included in the timing. The random-graph and complexity curves use CPU time per invocation; the error bars show the **sample standard deviation** over three repetitions, not confidence intervals or the dispersion of the 1,000 queries. The road-network table uses wall-clock time in the search loop divided by the number of queries. The raw CSV also retains both CPU and wall-clock times for the complete batch.

3.2 USA Road Network Inputs and Results

The NY graph has $n = 264346$ and $m = 733846$, while the BAY graph has $n = 321270$ and $m = 800172$. Both graphs use the complete set of 1,000 source-target queries. Each implementation runs one complete batch, for a total of $9 \times 2 \times 1000$ searches. The road-network data come from the 9th DIMACS Implementation Challenge [3].

Table 2: Mean wall-clock time per query. Speedup is the standard-library time divided by the time of the current implementation.

Implementation	NY/ms	BAY/ms	NY speedup	BAY speedup
std PQ	19.849	20.518	1.00	1.00
Binary heap	19.720	20.630	1.01	0.99
Segment tree	36.344	42.261	0.55	0.49
Fibonacci heap	44.803	51.184	0.44	0.40
Radix heap	12.621	13.873	1.57	1.48
PBDS pairing	26.985	30.088	0.74	0.68
PBDS thin	36.818	42.577	0.54	0.48
FIFO SPFA	249.409	353.306	0.08	0.06
SLF+LLL SPFA	7.267	9.258	2.73	2.22

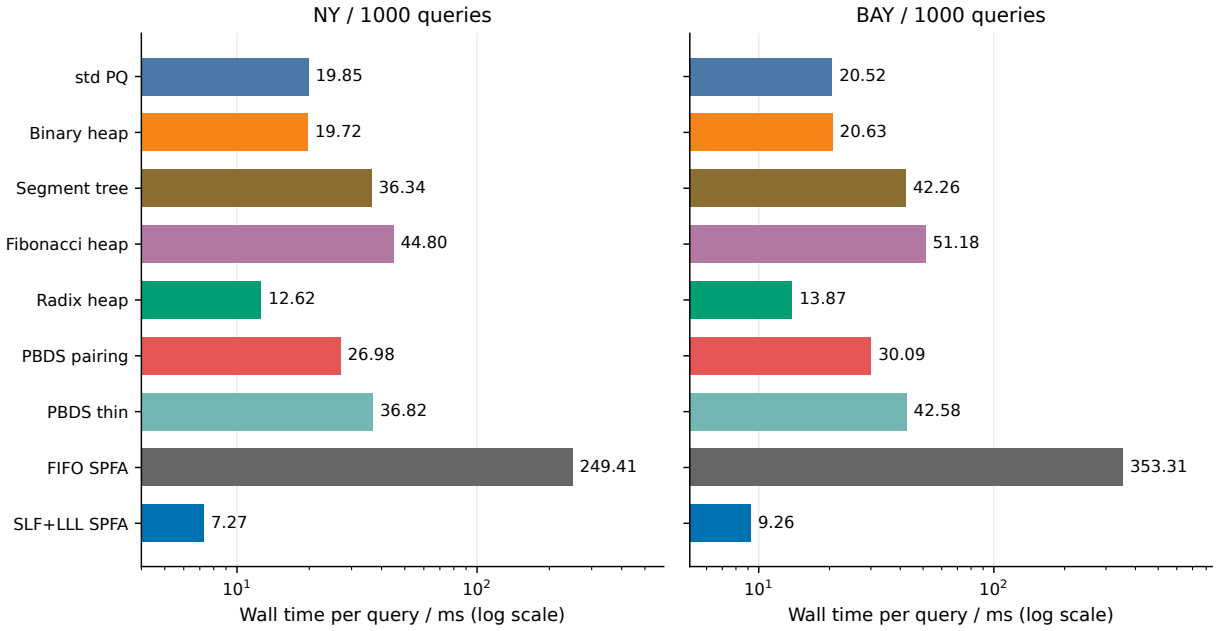


Figure 1: Mean query time on NY and BAY. The horizontal axis is logarithmic. Each measurement is one complete batch of 1,000 queries, and repeated-run variance was not estimated for the road networks.

On NY and BAY, the radix heap achieves speedups of approximately 1.57 and 1.48 over the standard-library queue, respectively. SLF+LLL achieves speedups of 2.73 and 2.22, while the Fibonacci heap takes 2.26 and 2.49 times as long as the standard-library queue. FIFO SPFA requires 249.409 and 353.306 ms/query and is much slower. These results apply only to the present inputs; they do not imply that FIFO SPFA is always slow or that SLF+LLL always has an advantage.

3.3 Random Graph Parameter Sweep

The random directed graphs contain 19 (n, m) configurations, with nine implementations per configuration, for a total of 171 cases. Multiedges are allowed, self-loops are excluded, weights lie in $[0, 200000]$, and the source is vertex 1. The graphs are not required to be connected. Reachability records from the same C++ generator show that, for the two graphs with $n = 163840$ ($m = 163839, 327678$) and the graph with $n = 10, m = 9$, only the source itself is reachable and the number of reachable arcs is zero. Timing for these cases mainly reflects initialization and invocation overhead rather than traversal of the whole graph. The complete reachability results are given in Appendix Listing 6; the raw timings are unchanged.

Table 3: Mean CPU time in ms for three representative random graphs. The corresponding arc counts are 327552, 327672, and 327676.

Implementation	$n = 2560$	$n = 40960$	$n = 81920$
std PQ	1.111	8.050	11.845
Binary heap	0.518	6.046	11.771
Segment tree	0.575	6.468	12.769
Fibonacci heap	0.808	12.751	26.152
Radix heap	0.424	3.246	6.252
PBDS pairing	0.736	9.412	18.288
PBDS thin	1.008	12.700	23.544
FIFO SPFA	0.646	3.710	5.074
SLF+LLL SPFA	0.555	2.960	3.751

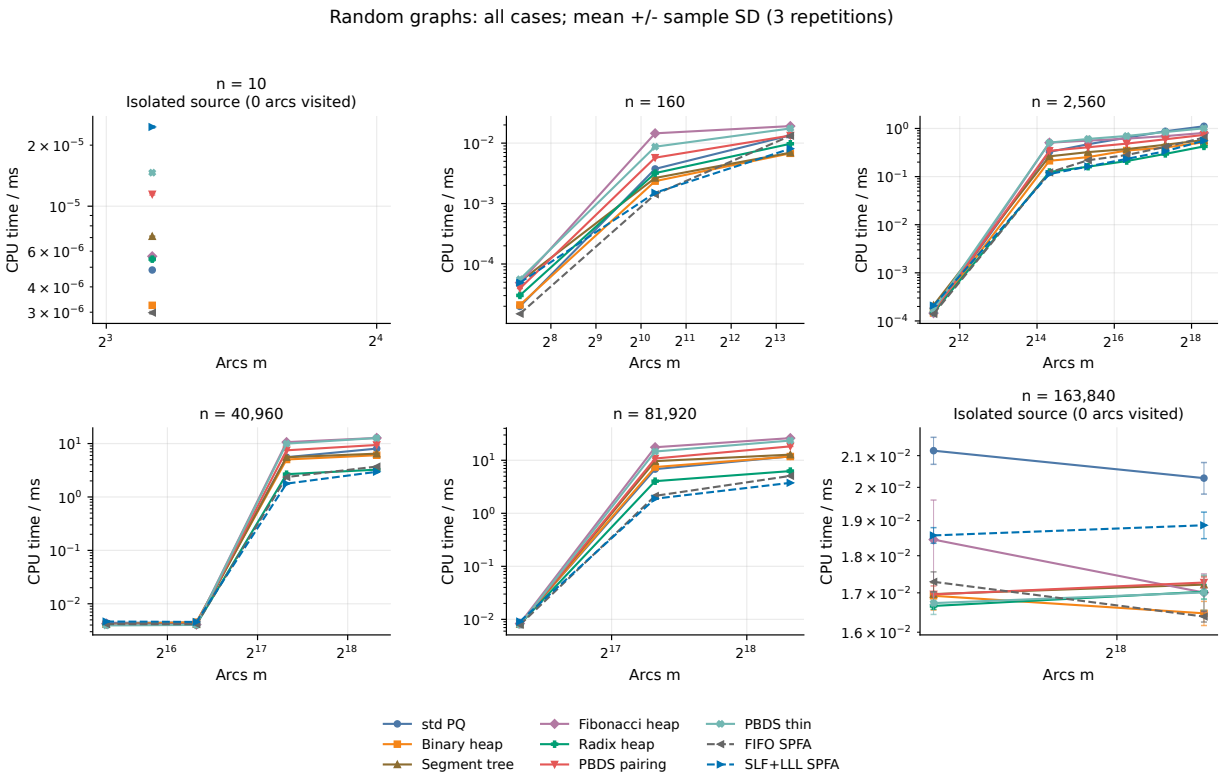


Figure 2: All random-graph configurations, faceted by a fixed vertex count so that only the arc count varies within each panel. Error bars show the sample standard deviation over three repetitions, and the vertical axis is logarithmic. Cases with an isolated source are marked in the panel titles.

3.4 Fixed-Density Experiments and Complexity Fitting

The fixed-density experiments use two graph families: a sparse family with $m = 4n$ and $n = 2^{10}, \ldots, 2^{17}$, and a dense family with $m = n(n-1)/4$ and $n = 2^7, \ldots, 2^{12}$. Nine implementations are measured at every size in each family, for a total of $9(8+6) = 126$ cases.

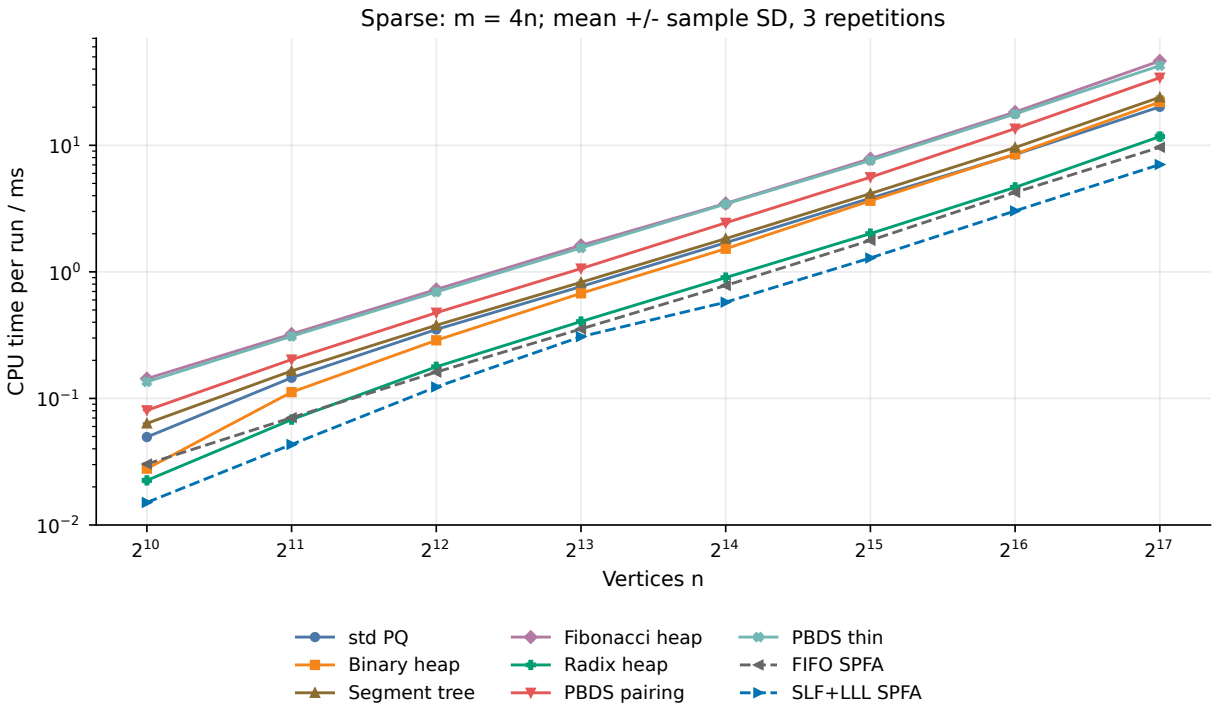


Figure 3: Mean CPU time for the sparse family, $m = 4n$. Error bars show the sample standard deviation over three repetitions.

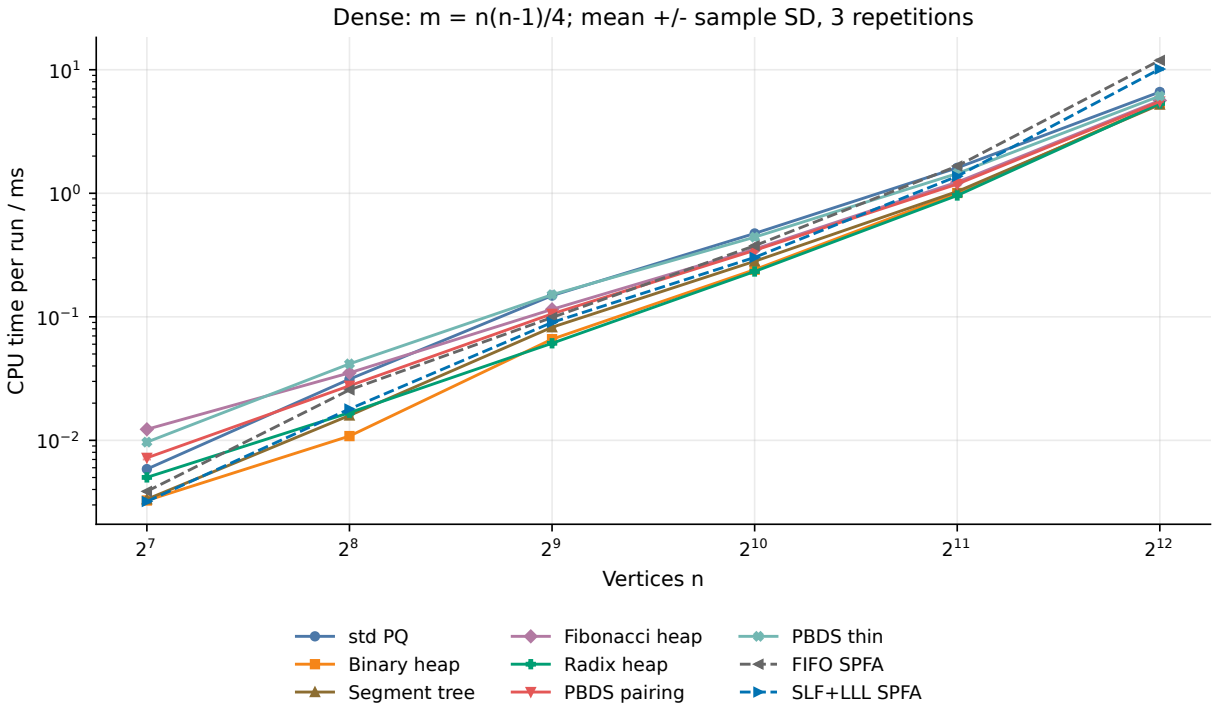


Figure 4: Mean CPU time for the dense family, $m = n(n-1)/4$. Error bars show the sample standard deviation over three repetitions.

Google Benchmark fits the models $f(n) = n \log_2 n$ and $f(n) = n^2$ using the origin-constrained least-squares form $T(n) \approx cf(n)$. In Table 4, the sparse coefficients are measured in ns/$(n \log_2 n)$

and the dense coefficients in ns/n^2; coefficients from the two columns cannot be compared directly. RMS is the normalized residual percentage defined by Google Benchmark and measures how well the model fits within the observed range.

Table 4: CPU coefficients and RMS for the fixed models. All values are exported from the CSV data.

Implementation	Sparse coef.	Sparse RMS	Dense coef.	Dense RMS
std PQ	8.806	9.1%	0.393	8.0%
Binary heap	9.444	16.4%	0.314	14.3%
Segment tree	10.361	13.4%	0.307	10.7%
Fibonacci heap	20.008	14.5%	0.334	5.9%
Radix heap	5.063	16.4%	0.313	14.1%
PBDS pairing	14.729	15.1%	0.327	6.8%
PBDS thin	18.562	10.9%	0.364	3.9%
FIFO SPFA	4.245	8.1%	0.690	23.2%
SLF+LLL SPFA	3.089	8.7%	0.586	24.3%

Applying the same model to SPFA is an empirical fit and is not used to infer its asymptotic bound. For dense graphs, the theoretical term for the Fibonacci heap is $m + n \log n = \Theta(n^2)$, whereas the usual upper bound for heap-based Dijkstra contains $m \log n$. At finite sizes, edge scanning may still dominate timing, so a good n^2 fit does not imply that the logarithmic factor has disappeared.

Chapter 4 Analysis and Comments

4.1 Time and Space Complexity

Table 5 considers one complete search from a given source. The graph itself uses $O(n + m)$ space and the distance array uses $O(n)$; queue space is listed separately. The standard-library version uses lazy deletion and may retain $O(m)$ entries, which differs substantially from implementations that keep only one heap entry per vertex.

Table 5: Cost and applicable conditions of each queue strategy

Strategy	Time bound or analysis	Applicable conditions	Queue space
Standard-library lazy deletion	$O(n + m \log(m + 1))$	Nonnegative weights; duplicate entries allowed	$O(m + 1)$
Indexed binary heap	$O((n + m) \log(n + 1))$	Nonnegative weights; direct decrease-key	$O(n)$
Segment-tree heap	$O((n + m) \log(n + 1))$	Fixed vertex-to-leaf mapping	$O(n)$
Fibonacci heap	$O(m + n \log(n + 1))$, amortized	Nonnegative weights; direct node access	$O(n)$
Two PBDS heaps	Depends on the selected library heap; not derived here	GCC extension and valid handles	$O(n)$
Radix heap	Bucket-migration cost $O(m + nB)$ for key bit width B	Nonnegative integer keys; monotone keys and capacity limits	$O(n + B)$
FIFO SPFA	Depends on successful relaxations and repeated enqueues; worst-case behavior can be poor	Only nonnegative weights are used here	Depends on the number of entries
SLF+LLL	Heuristic scheduling provides no Dijkstra-style worst-case guarantee	Only nonnegative weights are used here	$O(n)$

For the Fibonacci heap, all arc scans and successful decrease-key operations total $O(m)$, and each vertex is involved in at most one effective extract-min, for a total of $O(n \log n)$. On sparse graphs, this bound has the same order as that of a binary heap, but each Fibonacci-heap operation maintains more links, so the asymptotic difference does not necessarily yield a smaller constant.

The analysis of the radix heap relies on bucket indices decreasing during redistribution. If the largest valid key is bounded by U, then $B = \lceil \log_2(U + 1) \rceil$. The specialized radix design in the literature yields an $O(m + n \log C)$ bound for Dijkstra's algorithm, where C is the edge-weight upper bound [2]. The present implementation uses 32 buckets and compressed handles and does not exactly match all assumptions of that design, so the tighter bound is not adopted here.

This study did not collect peak-memory curves, hardware cache counters, the number of successful relaxations, or the number of bucket redistributions. The space analysis follows from the data structures and is not a measured memory result.

4.2 Observed Performance of the Fibonacci Heap

The NY and BAY road networks have average out-degrees of only about 2.78 and 2.49, respectively, and are therefore sparse. The Fibonacci heap reduces the asymptotic cost of decrease-key, but it must still consolidate roots of equal degree, maintain linked lists, and perform cascading

cuts. Although the nodes are allocated contiguously in an array, each operation follows node links, whereas most binary-heap state resides in compact arrays. More frequent link updates and poorer locality may explain part of the gap. This is an inference from the implementation structure and would require operation counts and performance-counter data to evaluate. No metrics that directly distinguish cache misses were collected.

In the dense family, edge scanning grows with n^2 and heap maintenance may account for a smaller fraction of the total time, reducing some gaps between queues. The largest dense case in this study has only $n = 4096$, which is insufficient to determine the eventual winner at large scale. Although the random graphs reach larger vertex counts, their sparsity is entirely different, so they cannot substitute for large-scale dense experiments.

4.3 Differences between the Radix Heap and SPFA

On nonnegative-integer road networks, the radix heap exploits the monotonicity of keys and avoids much of the sift-up and sift-down work in comparison heaps. It is the fastest of the seven Dijkstra queues in this study. It still requires valid key ranges and sufficient handle capacity, so it cannot directly replace a general priority queue for real-valued or non-monotone keys.

SLF+LLL is faster than every heap implementation on both networks, while FIFO SPFA is more than an order of magnitude slower. Changing only the scheduling order of SPFA thus changes performance substantially. The two implementations also use different memory strategies, duplicate entries versus an in-queue marker, so their gap cannot be attributed solely to SLF or LLL. Good performance on the random graphs does not guarantee efficiency on adversarial inputs.

4.4 Model Fitting and Measurement Limitations

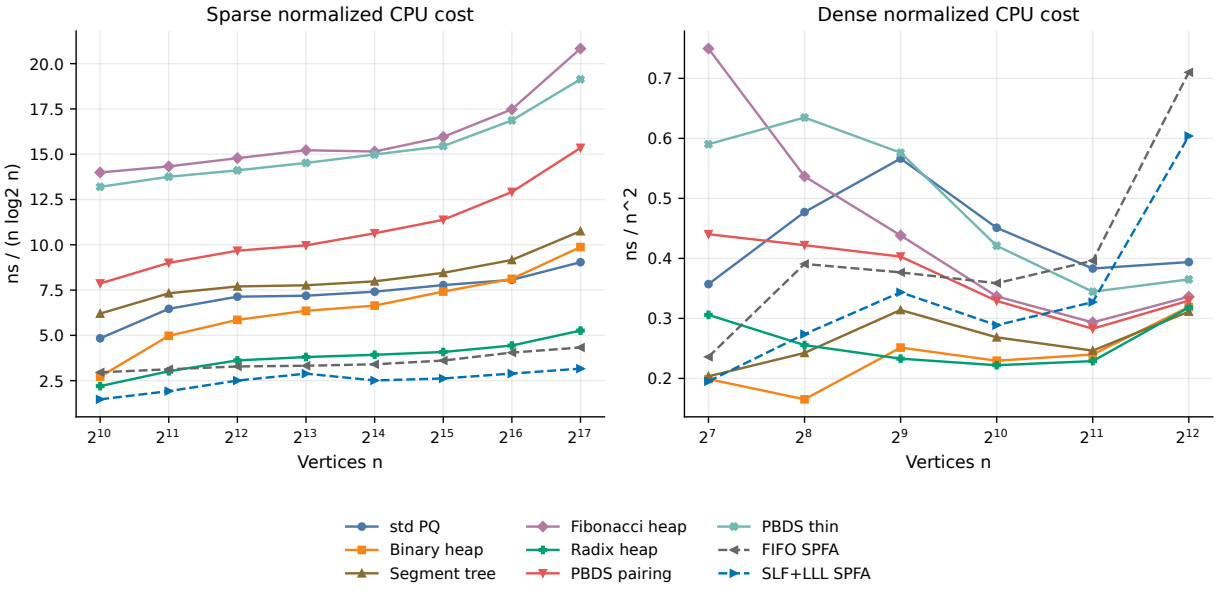


Figure 5: CPU time normalized by the prescribed models. The curves are not strictly constant; the figure is intended only to show model deviation and finite-size effects.

The complexity plots record timings at a finite set of discrete sizes. Changes in a normalized curve may come from the algorithm's operation count or from caching, compilation, clock frequency, or input reachability. A small RMS indicates only that the model fits the observed range; it does not rule out another model or establish a tight bound.

Three repetitions of the random and complexity experiments are insufficient to estimate tail behavior reliably. The road networks have only one batch and no per-query distribution, and the two network sizes are insufficient for a reliable scaling fit. WSL shares the host hardware, and CPU affinity and host load were not controlled, so absolute times serve only as a reference for this run.

4.5 Further Improvements

Further work should add independent road-network batches and per-query records, as well as larger graph sizes. It should also record the numbers of relaxations, outdated entries, root consolidations, and bucket redistributions, together with performance-counter data that can explain constant-factor costs. For true point-to-point queries, the implementations should first apply a common target-termination rule before introducing bidirectional search or preprocessing algorithms.

The amortized bounds did not determine the observed ordering. On the sparse road networks, the theoretical advantage of the Fibonacci heap did not translate into speed, while the radix heap and suitably scheduled SPFA were faster. This ordering remains specific to the present inputs and implementations.

Appendix A Source Code and Reproduction

A.1 Source Code Organization

The source code is located under `pq-test/` in the project. The main files relevant to this report are listed below.

Table 6: Main source files and their roles

File	Role
`csr-graph.hpp`, `dimacs-parser.hpp`	Shared graph representation and file parsing
`general-dijkstra.hpp`	Shared main Dijkstra loop
`heap-adaptor.hpp`	Timestamp interface, standard-library adaptor, and PBDS adaptors
`binary-heap.hpp`, `segment-tree-heap.hpp`	Binary heap and segment-tree heap
`fibonacci-heap.hpp`, `radix-heap.hpp`	Fibonacci heap and radix heap
`general-bellman-ford.hpp`	Two label-correcting algorithms
`dij-bench-v2.cpp`	Random-graph parameter sweep
`dij-bench-complexity.cpp`	Fixed-density complexity experiments
`dij-bench-dimacs.cpp`	Road-network query benchmark

The following excerpt shows the main loop in `general-dijkstra.hpp` and how the core logic is shared across different heap implementations.

```cpp
// Shared Dijkstra loop, excerpted without identifying file-header comments.
// The caller allocates the distance vector and supplies a reusable empty heap.
template <typename Heap>
inline void dijkstra(const Graph &g, uint32_t s,
                     std::vector<int64_t> &dis, Heap &heap) {
  // Leave headroom for distance addition on the validated input domain.
  constexpr int64_t INF = std::numeric_limits<int64_t>::max() / 4;
  // Initialization is part of every timed search.
  std::fill(dis.begin(), dis.end(), INF);
  heap.push_or_decrease(0, s);
  dis[s] = 0;
  while (!heap.empty()) {
    auto [d, u] = heap.pop();
    // Standard-library lazy insertion can leave outdated entries.
    if (d > dis[u]) continue;
    // CSR adjacency traversal is shared by all heap variants.
    for (auto [v, w] : g[u]) {
      auto nd = dis[u] + w;
      if (dis[v] > nd) {
        dis[v] = nd;
        // Insert on first discovery, decrease an existing key otherwise.
        heap.push_or_decrease(dis[v], v);
      }
    }
  }
  // This implementation deliberately drains the heap, even for p2p input.
}
```

Listing 5: Excerpt of the shared main loop

A.2 Random and Fixed-Density Graph Reachability Results

The following listing gives the vertex and reachable-arc counts for each graph size. These counts are used when interpreting the random-graph and complexity timings.

```
suite,scenario,n,m,reachable_vertices,reachable_arcs
dij-v2,random,10,9,1,0
dij-v2,random,160,159,3,2
dij-v2,random,160,1272,159,1260
dij-v2,random,160,10176,160,10176
dij-v2,random,2560,2559,3,2
dij-v2,random,2560,20472,2558,20456
dij-v2,random,2560,40944,2560,40944
dij-v2,random,2560,81888,2560,81888
dij-v2,random,2560,163776,2560,163776
dij-v2,random,2560,327552,2560,327552
dij-v2,random,40960,40959,2,1
dij-v2,random,40960,81918,2,1
dij-v2,random,40960,163836,40210,160865
dij-v2,random,40960,327672,40945,327535
dij-v2,random,81920,81919,1,0
dij-v2,random,81920,163838,65159,130698
dij-v2,random,81920,327676,80232,321087
dij-v2,random,163840,163839,1,0
dij-v2,random,163840,327678,1,0
complexity,sparse,1024,4096,999,3986
complexity,sparse,2048,8192,2015,8070
complexity,sparse,4096,16384,4007,16039
complexity,sparse,8192,32768,8047,32175
complexity,sparse,16384,65536,16057,64227
complexity,sparse,32768,131072,32124,128497
complexity,sparse,65536,262144,64217,256877
complexity,sparse,131072,524288,128488,513988
complexity,dense,128,4064,128,4064
complexity,dense,256,16320,256,16320
complexity,dense,512,65408,512,65408
complexity,dense,1024,261888,1024,261888
complexity,dense,2048,1048064,2048,1048064
complexity,dense,4096,4193280,4096,4193280
```

Listing 6: Reachability results for the random and fixed-density graphs

References

[1] M. L. Fredman, R. E. Tarjan. "Fibonacci heaps and their uses in improved network optimization algorithms." *Journal of the ACM*, 34(3), pp. 596–615, 1987. https://doi.org/10.1145/28869.28874.

[2] R. K. Ahuja, K. Mehlhorn, J. B. Orlin, R. E. Tarjan. "Faster algorithms for the shortest path problem." *Journal of the ACM*, 37(2), pp. 213–223, 1990. https://doi.org/10.1145/77600.77615.

[3] C. Demetrescu, A. V. Goldberg, D. S. Johnson. *9th DIMACS Implementation Challenge: Shortest Paths*. Official data directory: http://www.diag.uniroma1.it/challenge9/data/USA-road-d/. The original filenames for NY and BAY are `USA-road-d.NY.gr.gz` and `USA-road-d.BAY.gr.gz`, respectively. Source and archive information used in this project is recorded in `docs/DATA.md`.

[4] Google. *Google Benchmark User Guide*, vendored version 1.9.5. https://github.com/google/benchmark/blob/v1.9.5/docs/user_guide.md. The fitting and statistical fields used in this report can also be inspected in the local library implementation.

[5] Free Software Foundation. *Policy-Based Data Structures, libstdc++ Manual*. https://gcc.gnu.org/onlinedocs/libstdc++/manual/policy_data_structures.html.

Declaration

We hereby declare that all work done in this project, "Implementation and Evaluation of Dijkstra's Algorithm
with Different Priority Queues," is of our independent effort as a group.